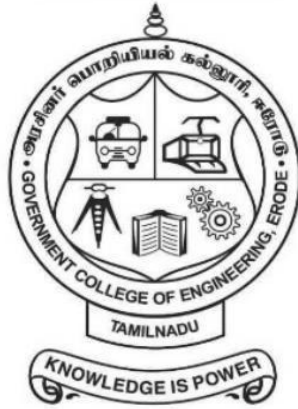


**GOVERNMENT COLLEGE OF ENGINEERING,
ERODE - 638 316**



RECORD NOTE BOOK

Reg. No. _____

Certified that this is the bonafide record of work done by Selvan / Selvi

_____ of the Sixth Semester of B.TECH-

INFORMATION TECHNOLOGY during the Academic year 2023 – 2024 in the **CCS353-**

MULTIMEDIA DATA COMPRESSION AND STORAGE.

Staff-in-charge

Head of the Department

Submitted for the University Practical Examination on _____ at the Government
College of Engineering, Erode – 638316.

Internal Examiner

External Examiner

S.NO	DATE	EXPERIMENT	PG. NO	SIGNATURE
1		Construct Huffman codes for given symbol probabilities.		
2		Encode run lengths with fixed-length code.		
3		Lempel-Ziv algorithm for adaptive variable-length encoding		
4		Compress the given word using arithmetic coding based on the frequency of the letters.		
5		Write a shell script, which converts all images in the current directory in JPEG.		
6		Write a program to split images from a video without using any primitives.		
7		Create a photo album of a trip by applying appropriate image dimensions and format.		
8		Write the code for identifying the popularity of content retrieval from media server.		
9		Write the code for ensuring data availability in disks using strip based method.		
10		Program for scheduling requests for data streams.		

Ex.No:01	Construct Huffman codes for given Symbol probabilities.
Date:	

Aim:

To write a program to construct Huffman codes for given symbol probabilities.

Program:

```
# Huffman coding python implementation
from collections import Counter
class NodeTree(object):
    def __init__(self, left=None, right=None):
        self.left = left
        self.right = right

    def children(self):
        return self.left, self.right

    def __str__(self):
        return self.left, self.right

def huffman_code_tree(node, binString=""):
    if type(node) is str:
        return {node: binString}
    (l, r) = node.children()
    d = dict()
    d.update(huffman_code_tree(l, binString + '0'))
    d.update(huffman_code_tree(r, binString + '1'))
    return d

def make_tree(nodes):
    while len(nodes) > 1:
        (key1, c1) = nodes[-1]
        (key2, c2) = nodes[-2]
```

```
nodes = nodes[:-2]

node = NodeTree(key1, key2)
nodes.append((node, c1 + c2))
nodes = sorted(nodes, key=lambda x: x[1], reverse=True)
return nodes[0][0]

if __name__ == '__main__':
    string = 'BCAADDCCACACACABABABABAB'
    freq = dict(Counter(string))
    freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)
    node = make_tree(freq)
    encoding = huffman_code_tree(node)
    for i in encoding:
        print(f'{i} : {encoding[i]}')
```

Output:

```
A : 0
B : 10
D : 110
C : 111
```

Result:

Thus the program has been written and executed successfully.

EX.No:02	Encode run lengths with fixed_length code.
Date:	

Aim:

To write a program to Encode run lengths with fixed_length code.

Program:

```
# run length encoding

def printRLE(st):
    n = len(st)
    i=0
    while i < n- 1:

        #Count occurrences of
        #current character
        count = 1
        while (i < n 1 and st[i] == st[i + 1]):
            count+=1
            i += 1

        #Print character and its count
        print(st[i] + str(count), end = "")
```

Output:

w4a3d1e1x6y1w3

Result:

Thus the program has been written and executed successfully.

Ex.No:03	Lempel-Ziv algorithm for adaptive variable-length encoding.
Date:	

Aim:

To write a program to Lempel-Ziv algorithm for adaptive variable-length encoding.

Program:

```
#L LZ
input_str = 'AAAABBCDEABCDABCAAABCDEEEEEECBBBBBBDDAAE'
keys_dict = {}
ind = 0
inc = 1
while True:
    if not (len(input_str) >= ind+inc):
        break
    sub_str = input_str[ind:ind + inc]
    print (sub_str,ind,inc)
    if sub_str in keys_dict:
        inc += 1
    else:
        keys_dict[sub_str] = 0
        ind += inc
        inc = 1
    # print 'Adding %s' %sub_str
print (list(keys_dict))
```

Output:

```
A 0 1
A 1 1
AA 1 2
A 3 1
AB 3 2
B 5 1
C 6 1
D 7 1
E 8 1
A 9 1
AB 9 2
ABC 9 3
```


D 12 1

DA 12 2

B 14 1

BC 14 2

A 16 1

AA 16 2

AAA 16 3

B 19 1

BC 19 2

BCD 19 3

E 22 1

EE 22 2

E 24 1

EE 24 2

EEE 24 3

E 27 1

EC 27 2

B 29 1

BB 29 2

B 31 1

BB 31 2

BBB 31 3

B 34 1

BD 34 2

D 36 1

DA 36 2

DAA 36 3

E 39 ['A', 'AA', 'AB', 'B', 'C', 'D', 'E', 'ABC', 'DA', 'BC', 'AAA', 'BCD', 'EE', 'EEE', 'EC', 'BB', 'BBB', 'BD', 'DAA']

Result:

Thus the program has been written and executed successfully.

Ex.No:04	Compress the given word using arithmetic coding based on the frequency of the letters.
Date:	

Aim:

To write a program to compress the given word using arithmetic coding based on the frequency of the letters.

Program(A):

```
def get_unique_char(message): # O(n^2)
    unique_char = []
    for character in message: # O(n)
        unique = True
        # check if the character has already been added to unique_char list
        for e in unique_char: # O(n)
            if e == character:
                # change unique to False if the character already exist in unique_char list
                unique = False
                break
        if unique:
            unique_char.append(character) # O(1) amortized
    return unique_char

def get_frequency(message, unique_char): # O(n^2)
    # use a dictionary where
    # the key is the unique character
    # the value is the frequency
    frequency = {}
    for character in unique_char: # O(n)
        char_freq = 0
        # count the number of occurrences of the current character
        for e in message: # O(n)
```

```

    if character == e:

        # increase char_freq by 1 every time a character in message

        # is the same as a character in the unique_char list

        char_freq += 1

    frequency[character] = char_freq # Average case O(1) or Amortized worst case O(n)

return frequency

def get_occurring_probability(message, frequency): # O(n)

    # use a dictionary where

    # the key is the unique character

    # the value is the probability of occurrence

    probability = {}

    message_length = len(message)

    for key, value in frequency.items(): # O(n)

        # probability of occurrence of a unique character = frequency/message_length

        probability[key] = value / message_length

    return probability

def get_cumulative_sum(lower_bound, upper_bound, probability_ls): # O(n) (where append to a
list takes O(1) amortized)

    cumulative_sum = [lower_bound]

    diff_btw_two_bounds = upper_bound - lower_bound

    char_lower_bound = lower_bound

    for probability in probability_ls: # O(n)

        char_upper_bound = char_lower_bound + (diff_btw_two_bounds * probability)

        cumulative_sum.append(char_upper_bound) # O(1) amortized

        char_lower_bound = char_upper_bound

    return cumulative_sum

def associate_key_with_interval(cumulative_sum, unique_char):

    # O(n) (where adding to a dictionary O(1) average case)

    # use a dictionary where

    # the key is the unique character

    # the value is a list of length 2 where

```

```

# - the first element is the lower bound
# - the second element is the upper bound
interval = {}
i = 0
j = 0
while i < len(cumulative_sum) - 1: # O(n)
    key = unique_char[j]
    lower_bound = cumulative_sum[i]
    upper_bound = cumulative_sum[i + 1]
    interval[key] = [lower_bound, upper_bound] # Average case O(1) or Amortized worst case O(n)
    i += 1
    j += 1
return interval

# get_tag() takes O(n^2) (if get_cumulative_sum() and associate_key_with_interval() take O(n))
def get_tag(probability, unique_char, message):
    # put all values from probability dictionary into a probability list
    probability_ls = []
    for key, value in probability.items(): # O(n)
        probability_ls.append(value) # O(1) amortized
    # then use the probability list to calculate cumulative sum of probability_ls
    # initially, the lower bound is 0.0 and the upper bound is 1.0
    cumulative_sum = get_cumulative_sum(0.0, 1.0, probability_ls) # O(n) (where append to a list takes O(1) amortized)
    print('Cumulative sum for interval [0, 1): ', cumulative_sum)
    # associate each key with its interval
    interval_dict = associate_key_with_interval(cumulative_sum,
                                                unique_char) # O(n) (where adding to a dictionary O(1) average case)

    # get the tag
    tag = 0.0
    for character in message: # O(n)

```

```

# get the interval of the current character (narrow down the interval)
char_interval = interval_dict.get(character) # Average case O(1) or Amortized worst case O(n)

# get the lower and upper bound of the interval of the current character
char_lower_bound = char_interval[0]
char_upper_bound = char_interval[1]

# calculate the tag: tag = average of lower and upper bound
# the tag is recalculated until the last element in the message is reached
tag = (char_lower_bound + char_upper_bound) / 2.0

# every time the interval is narrowed down:
# - get the new cumulative sum for the new interval
# - each key will have a new lower and upper bound in the new interval
cumulative_sum = get_cumulative_sum(char_lower_bound, char_upper_bound,
                                     probability_ls) # O(n) (where append to a list takes O(1))

interval_dict = associate_key_with_interval(cumulative_sum,
                                           unique_char) # O(n) (where adding to a dictionary O(1) average case)

return tag

def concatenate_char(ls): # O(n)
    string = ""
    for e in ls: # O(n)
        string += e
    return string

def arithmetic_encoding(message): # O(n^2)
    # generate a list that contains unique characters in the message
    unique_char = get_unique_char(message) # O(n^2)
    print('Unique char in the message: ', unique_char)

    # get frequency of occurrences of all unique characters in the message
    frequency = get_frequency(message, unique_char) # O(n^2)
    print('Frequency of each unique character: ', frequency)

    # get probability of occurrence of all unique characters in the message
    probability = get_occurring_probability(message, frequency) # O(n)
    print('Occurring probability of each unique character: ', probability)

```

```
# get the tag: get_tag() takes  $O(n^2)$  (if get_cumulative_sum() and associate_key_with_interval() take  $O(n)$ )
```

```
tag = get_tag(probability, unique_char, message) #  $O(n^2)$ 
```

```
return tag, probability
```

```
def arithmetic_decoding(probability, message_length, tag): #  $O(n^2)$ 
```

```
# put all values from probability dictionary into a probability list
```

```
# put all keys from probability dictionary into a unique_char list
```

```
probability_ls = []
```

```
unique_char = []
```

```
for key, value in probability.items(): #  $O(n)$ 
```

```
    probability_ls.append(value) #  $O(1)$  amortized
```

```
    unique_char.append(key) #  $O(1)$  amortized
```

```
# then use the probability list to calculate cumulative sum of probability_ls
```

```
# initially, the lower bound is 0.0 and the upper bound is 1.0
```

```
cumulative_sum = get_cumulative_sum(0.0, 1.0, probability_ls) #  $O(n)$  (where append to a list takes  $O(1)$  amortized)
```

```
# associate each key with its interval
```

```
interval_dict = associate_key_with_interval(cumulative_sum,  
                                           unique_char) #  $O(n)$  (where adding to a dictionary  $O(1)$  average case)
```

```
i = 0
```

```
message_char_ls = []
```

```
current_lower_bound = 0.0
```

```
current_upper_bound = 1.0
```

```
while i < message_length: #  $O(n)$ 
```

```
    for key, value in interval_dict.items(): #  $O(n)$ 
```

```
        # get the interval of the current character (key)
```

```
        lower_bound = value[0]
```

```
        upper_bound = value[1]
```

```
        # check if tag is within the interval of the current character (key)
```

```
        if (tag > lower_bound) and (tag < upper_bound):
```

```

        # narrow down the interval

        current_lower_bound = lower_bound

        current_upper_bound = upper_bound

        # add the character to message_char_ls if tag is within the interval of the current
character (key)

        message_char_ls.append(key) # O(1) amortized

        break

    # every time the interval is narrowed down:

    # - get the new cumulative sum for the new interval

    # - each key will have a new lower and upper bound in the new interval

    cumulative_sum = get_cumulative_sum(current_lower_bound, current_upper_bound,
                                         probability_ls) # O(n) (where append to a list takes O(1) amortized)

    interval_dict = associate_key_with_interval(cumulative_sum,
                                               unique_char) # O(n) (where adding to a dictionary O(1) average case)

    i += 1

return concatenate_char(message_char_ls)

def run_arithmetic_coding():

    message = 'OpenGenus'

    message_len = len(message)

    print('The message is ', message)

    print('The length of the message is', message_len)

    tag, probability = arithmetic_encoding(message)

    print('The tag for ', message, ' is ', tag)

    decoded_msg = arithmetic_decoding(probability, message_len, tag)

    print('Decode the message using probability (coding model), message length, and the
generated tag: ', decoded_msg)

if __name__ == '__main__':

    run_arithmetic_coding()

```

Output(A):

The message is OpenGenus

The length of the message is 9

Unique char in the message: ['O', 'p', 'e', 'n', 'G', 'u', 's']

Frequency of each unique character: {'O': 1, 'p': 1, 'e': 2, 'n': 2, 'G': 1, 'u': 1, 's': 1}

Occurring probability of each unique character: {'O': 0.1111111111111111, 'p': 0.1111111111111111, 'e': 0.2222222222222222, 'n': 0.2222222222222222, 'G': 0.1111111111111111, 'u': 0.1111111111111111, 's': 0.1111111111111111}

Cumulative sum for interval [0, 1): [0.0, 0.1111111111111111, 0.2222222222222222, 0.4444444444444444, 0.6666666666666666, 0.7777777777777777, 0.8888888888888888, 1.0]

The tag for OpenGenus is 0.016739628347327812

Decode the message using probability (coding model), message length, and the generated tag: OpenGenus

Program(B):

```
import string
import random
from collections import Counter
import time

# Arithmetic Encoding
def ac_encode(txt):
    res = Counter(txt)

    # characters
    chars = list(res.keys())

    # frequency of characters
    freq = list(res.values())

    probability = []

    for i in freq:
        probability.append(i / len(txt))

    print(chars)
    print(probability)

    high = 1.0
    low = 0.0

    for c in txt:
        diff = high - low
```



```

index = chars.index(c)

for i in range(index):
    high = low + diff * probability[i]
    low = high
high = low + diff * probability[index]
print(f'char {c} -> Low: {low} High: {high}')
tag = (low+high)/2.0
print('Input: ' + txt)
print(str(low) + '< codeword <' + str(high))
print('codeword = ' + str(tag))
with open('encode.ac', 'w') as fw:
    for i in chars:
        fw.write(i + ' ')
    fw.write('\n')
    for i in probability:
        fw.write(str(i) + ' ')
    fw.write('\n')
    fw.write(str(tag))
return chars, probability, tag

```

Arithmetic Decoding

```

def ac_decode(chars, probability, tag):
    high = 1.0
    low = 0.0
    output = ""
    c = ""
    while (c != '$'):
        diff = high - low
        for i in range(len(chars)):
            high = low + diff * probability[i]
            if low < tag < high:

```

```

        break

    else:

        low = high

    c = chars[i]

    output += c

return output


def arithmetic_coding(input):

    if '$' in input:

        input = input[0:input.index('$')]

    if input[-1] != '$':

        input += '$'

    print('Input: ' + input)


    start = time.time()

    (chars, probability, tag) = ac_encode(input)

    output = ac_decode(chars, probability, tag)

    end = time.time()

    print('Decode: ' + output)

    print('does match : ' + str(input == output))

    print(f"Total Time: {end - start} sec\n\n")

    return input == output


##### INPUT #####

# Random String , 100 test case

count = 0

testcase = 10

for i in range(testcase):

    # generating string

    letters = string.ascii_uppercase

    random_txt = ''.join(random.choice(letters) for i in range(13)) + '$'

```

```

flag = arithmetic_coding(random_txt)

if flag:

    count += 1

print(f"Total Test: {testcase}")

print(f"Succcess: {count}")

#-----

# User given specific data

# Please use small string (less than 13 characters)

txt = "BANGLADESH$"

arithmetic_coding(txt)

```

Output(B):

Input: DZJHDDDDTCRWLP\$

['D', 'Z', 'J', 'H', 'T', 'C', 'R', 'W', 'L', 'P', '\$']

[0.2857142857142857, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142, 0.07142857142857142]

char D -> Low: 0.0 High: 0.2857142857142857

char Z -> Low: 0.08163265306122448 High: 0.1020408163265306

char J -> Low: 0.08892128279883381 High: 0.09037900874635567

char H -> Low: 0.0895460224906289 High: 0.08965014577259475

char D -> Low: 0.0895460224906289 High: 0.08957577199976201

char D -> Low: 0.0895460224906289 High: 0.08955452235038122

char D -> Low: 0.0895460224906289 High: 0.0895484510219867

char T -> Low: 0.08954723675630781 High: 0.08954741022283337

char C -> Low: 0.0895473358800367 High: 0.08954734827050281

char R -> Low: 0.08954734384533636 High: 0.08954734473036965

char W -> Low: 0.08954734447750302 High: 0.08954734454071969

char L -> Low: 0.08954734452717322 High: 0.0895473445316887

char P -> Low: 0.08954734453104363 High: 0.08954734453136616

char \$ -> Low: 0.08954734453134311 High: 0.08954734453136615

Input: DZJHDDDDTCRWLP\$

0.08954734453134311< codeword <0.08954734453136615

codeword = 0.08954734453135463

Decode: DZJHDDTCRWLP\$

does match : True

Total Time: 0.005059957504272461 sec

Result:

Thus the program has been written and executed successfully.

Ex.No:05	Write a shell script, which converts all images in the current directory in JPEG.
Date:	

Aim:

To write a shell script, which converts all images in the current directory in JPEG.

Program:

Create a directory "images"

```
$ mkdir images
```

```
$ cd images
```

Run the update command:

```
$ sudo apt update
```

Install "imagemagick"

```
$ sudo apt install imagemagick
```

Command to download a .png file in Linux subsystem:

```
$ wget
```

```
https://upload.wikimedia.org/wikipedia/commons/thumb/a/a5/Instagram_icon.png/1200px-Instagram_icon.png
```

Command to create a shell script:

```
$ nano pngtojpg.sh
```

Paste the following code :

```
#!/bin/bash
```

```
# Check if ImageMagick is installed
```

```
if ! command -v convert &> /dev/null; then
```

```
    echo "Error: ImageMagick is not installed. Please install it first."
```

```
    exit 1
```

```
fi
```

```
# Convert images to JPEG
```

```
for file in *.{png,jpg,jpeg,gif,bmp}; do
```

```
    if [ -f "$file" ]; then
```

```
        filename="${file%.*}"
```

```
        extension="${file##*.}"
```

```
if [ "$extension" != "jpg" ] && [ "$extension" != "jpeg" ]; then
    convert "$file" "${filename}.jpg"
    echo "Converted $file to ${filename}.jpg"
fi
fi
done
echo "Conversion completed."
```

To save and exit:

Ctrl+s

Ctrl+x

Make sure that both the script and png file is located in the same directory!

Now run the shell script:

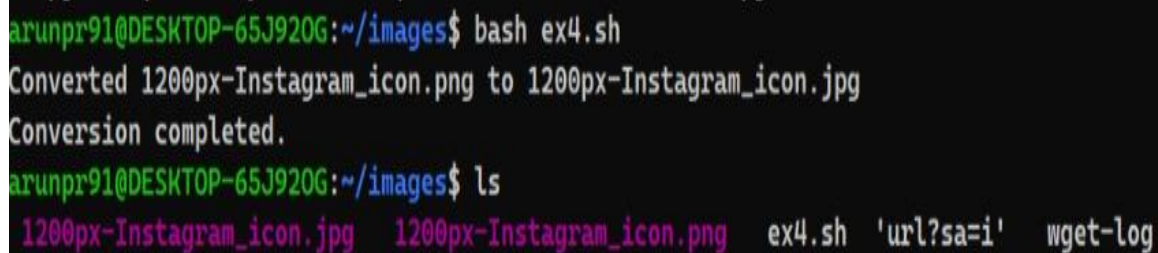
```
$ bash pngtojpg.sh
```

The png file will be converted into Jpg file :

```
$ ls
```

will show both png and jpg files in same directory.

Output:



```
arunpr91@DESKTOP-65J920G:~/images$ bash ex4.sh
Converted 1200px-Instagram_icon.png to 1200px-Instagram_icon.jpg
Conversion completed.
arunpr91@DESKTOP-65J920G:~/images$ ls
1200px-Instagram_icon.jpg 1200px-Instagram_icon.png ex4.sh 'url?sa=i' wget-log
```

Result:

Thus the program has been written and executed successfully.

Ex.No:06	Write a program to split images from a video without using any primitives.
Date:	

Aim:

To write a program to split images from a video without using any primitives.

Install cv2 using the following command:

```
>pip install opencv-python
```

Program:

```
# Importing all necessary libraries
```

```
import cv2
```

```
import os
```

```
# Read the video from specified path
```

```
cam = cv2.VideoCapture("E:\\MDCS\\LAB\\EXP6\\vid.mp4")
```

```
try:
```

```
    # creating a folder named data
```

```
    if not os.path.exists('data'):
```

```
        os.makedirs('data')
```

```
# if not created then raise error
```

```
except OSError:
```

```
    print ('Error: Creating directory of data')
```

```
# frame
```

```
currentframe = 0
```

```
while(True):
```

```
    # reading from frame
```

```
    ret,frame = cam.read()
```

```
    if ret:
```

```
        # if video is still left continue creating images
```

```
        name = './data/frame' + str(currentframe) + '.jpg'
```

```
        print ('Creating...' + name)
```

```
        # writing the extracted images
```

```
        cv2.imwrite(name, frame)
```

```
# increasing counter so that it will
# show how many frames are created
currentframe += 1

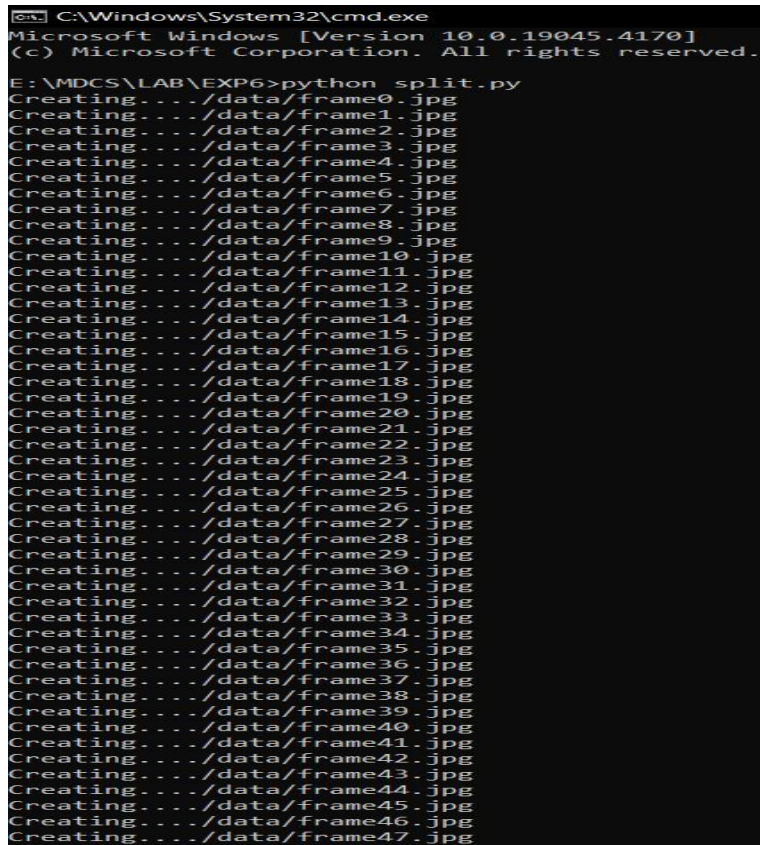
else:
    break
```

```
# Release all space and windows once done
cam.release()
cv2.destroyAllWindows()
```

Run the command:
>python filename.py

Then a new directory will be created which will contain all the images.
Thus the video is converted/split into images.

Output:



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

E:\MDCS\LAB\EXP6>python split.py
Creating.../data/frame0.jpg
Creating.../data/frame1.jpg
Creating.../data/frame2.jpg
Creating.../data/frame3.jpg
Creating.../data/frame4.jpg
Creating.../data/frame5.jpg
Creating.../data/frame6.jpg
Creating.../data/frame7.jpg
Creating.../data/frame8.jpg
Creating.../data/frame9.jpg
Creating.../data/frame10.jpg
Creating.../data/frame11.jpg
Creating.../data/frame12.jpg
Creating.../data/frame13.jpg
Creating.../data/frame14.jpg
Creating.../data/frame15.jpg
Creating.../data/frame16.jpg
Creating.../data/frame17.jpg
Creating.../data/frame18.jpg
Creating.../data/frame19.jpg
Creating.../data/frame20.jpg
Creating.../data/frame21.jpg
Creating.../data/frame22.jpg
Creating.../data/frame23.jpg
Creating.../data/frame24.jpg
Creating.../data/frame25.jpg
Creating.../data/frame26.jpg
Creating.../data/frame27.jpg
Creating.../data/frame28.jpg
Creating.../data/frame29.jpg
Creating.../data/frame30.jpg
Creating.../data/frame31.jpg
Creating.../data/frame32.jpg
Creating.../data/frame33.jpg
Creating.../data/frame34.jpg
Creating.../data/frame35.jpg
Creating.../data/frame36.jpg
Creating.../data/frame37.jpg
Creating.../data/frame38.jpg
Creating.../data/frame39.jpg
Creating.../data/frame40.jpg
Creating.../data/frame41.jpg
Creating.../data/frame42.jpg
Creating.../data/frame43.jpg
Creating.../data/frame44.jpg
Creating.../data/frame45.jpg
Creating.../data/frame46.jpg
Creating.../data/frame47.jpg
```

Result:

Thus the program has been written and executed successfully.

Ex.No:07

Creating a Photo Album of Different Image Format.

Date:

Aim:

To create a photo album of a college trip by applying different image dimensions and formats.

Software Required:

- Microsoft Photos
- Canva

Steps:

Step 1: Select an image from your college trip.

Step 2: Open the image in your favourite image editor. Let's go with Windows default photo viewer Photos.

Step 3: Choose the edit option in Microsoft's Photos, then try different image dimensions.

Step 4: After choosing the right image dimension, save the image in your desired format, in our case it is JPEG.

Step 5: For printed albums, the preferred size dimensions are 4X6 inches, 5X7 inches, or 8X10 inches. While for a digital try saving the image in 1920X1080 pixels.

Step 6: Canva can be used to add some additional details to the image. While using Canva you can change the format of the image when saving the image.

Step 7: Canva is a great tool for image editing and other image related works.

Original Image:



Image Format : .JPG

Image Dimension : 612x409

Image Size : 52 KB

Image After Changing Dimension:



Image Format : .JPG

Image Dimension : 545x409

Image Size : 65 KB

Here are the screenshots for changing the dimensions of the image.

Changing the image format:

- Changing the format of the image from JPG to PNG. Canva comes handy in this situation.
- Using Canva (which is online based image editing tool) you can change the format of the image.
- Canva can be used to change the dimensions of the image.

Image After Chaning Format:



Image Format : .PNG

Image Dimensions : 612x409

Image Size : 418 KB

Result:

Thus the creation of album with a image of different dimension and format is done successfully.

Ex.No:08	Write the code for identifying the popularity of content retrieval from media server.
Date:	

Aim:

To write the code for identifying the popularity of content retrieval from media server.

Program:

```
class MediaServer:
```

```
    def __init__(self):
```

```
        self.popularity = {}
```

```
    def track_request(self, content_id):
```

```
        if content_id in self.popularity:
```

```
            self.popularity[content_id] += 1
```

```
        else:
```

```
            self.popularity[content_id] = 1
```

```
    def get_popularity(self, content_id):
```

```
        return self.popularity.get(content_id, 0)
```

```
    def get_top_content(self, n=10):
```

```
        sorted_popularity = sorted(self.popularity.items(), key=lambda x: x[1], reverse=True)
```

```
        return sorted_popularity[:n]
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    server = MediaServer()
```

```
# Simulate requests
```

```
requests = [1, 2, 3, 1, 2, 4, 1, 3, 2, 1, 4, 5, 1, 2, 3]
```

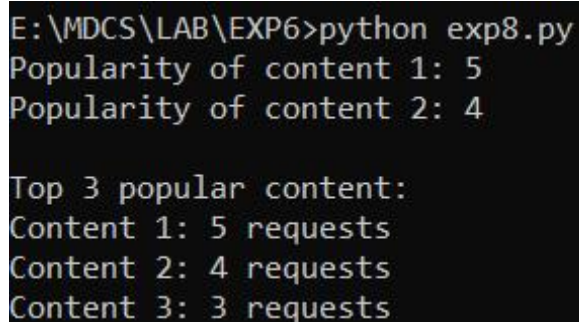
```
# Track requests

for request in requests:
    server.track_request(request)

# Get popularity of specific content
print("Popularity of content 1:", server.get_popularity(1))
print("Popularity of content 2:", server.get_popularity(2))

# Get top N popular content
top_content = server.get_top_content(3)
print("\nTop 3 popular content:")
for content_id, popularity in top_content:
    print(f"Content {content_id}: {popularity} requests")
```

Output:



```
E:\MDCS\LAB\EXP6>python exp8.py
Popularity of content 1: 5
Popularity of content 2: 4

Top 3 popular content:
Content 1: 5 requests
Content 2: 4 requests
Content 3: 3 requests
```

Result:

Thus the program has been written and executed successfully.

Ex.No:09	Write the code for ensuring data availability in disks using strip based method.
Date:	

Aim:

To write the code for ensuring data availability in disks using strip based method.

Program:

class Disk:

```

def __init__(self, name):
    self.name = name
    self.data = {}

def write_data(self, block_number, data):
    self.data[block_number] = data

def read_data(self, block_number):
    return self.data.get(block_number, None)

def is_available(self, block_number):
    return block_number in self.data

```

class DiskArray:

```

def __init__(self, num_disks):
    self.disks = [Disk(f"Disk_{i}") for i in range(num_disks)]
    self.num_disks = num_disks

def write_data(self, block_number, data):
    if len(data) != self.num_disks:
        raise ValueError("Data length must match the number of disks")
    for i in range(self.num_disks):
        self.disks[i].write_data(block_number, data[i])

def read_data(self, block_number):
    data = []
    for i in range(self.num_disks):
        data.append(self.disks[i].read_data(block_number))

```

```
        return data

def is_available(self, block_number):

    for disk in self.disks:

        if disk.is_available(block_number):

            return True

    return False


# Example usage:

disk_array = DiskArray(3) # Creating a disk array with 3 disks


# Writing data to disk array

data_to_write = ["Data1", "Data2", "Data3"]

disk_array.write_data(0, data_to_write)


# Checking availability of block 0

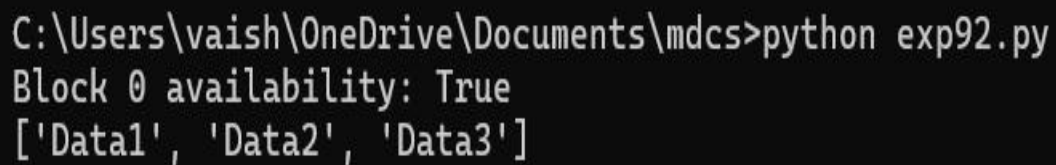
print("Block 0 availability:", disk_array.is_available(0)) # Output: Block 0 availability: True


# Reading data from disk array

read_data = disk_array.read_data(0)

print(read_data) # Output: ['Data1', 'Data2', 'Data3']
```

Output:



```
C:\Users\vaish\OneDrive\Documents\mdcs>python exp92.py
Block 0 availability: True
['Data1', 'Data2', 'Data3']
```

Result:

Thus the program has been written and executed successfully.

Ex.No:10	Program for scheduling requests for data streams.
Date:	

Aim:

To write a program for scheduling requests for data streams.

Program:

```
import heapq
```

```
import time
```

```
class DataStreamScheduler:
```

```
    def __init__(self):
```

```
        self.queue = [] # Priority queue to store requests
```

```
    def schedule_request(self, request, priority):
```

```
        """Schedule a new request"""
```

```
        heapq.heappush(self.queue, (priority, time.time(), request)) # Priority, timestamp, request
```

```
    def process_requests(self):
```

```
        """Process requests in priority order"""
```

```
        while self.queue:
```

```
            priority, _, request = heapq.heappop(self.queue) # Get the request with highest priority
```

```
            print(f"Processing request: {request}")
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
    scheduler = DataStreamScheduler()
```

```
# Schedule some requests
```

```
scheduler.schedule_request("Data stream request 1", priority=3)
```

```
scheduler.schedule_request("Data stream request 2", priority=1)
```

```
scheduler.schedule_request("Data stream request 3", priority=2)
```

```
# Process requests
```

```
scheduler.process_requests()
```

Output:

```
C:\Users\vaish\OneDrive\Documents\mdcs>python mdcs1.py  
Processing request: Data stream request 2  
Processing request: Data stream request 3  
Processing request: Data stream request 1
```

Result:

Thus the program has been written and executed successfully.